

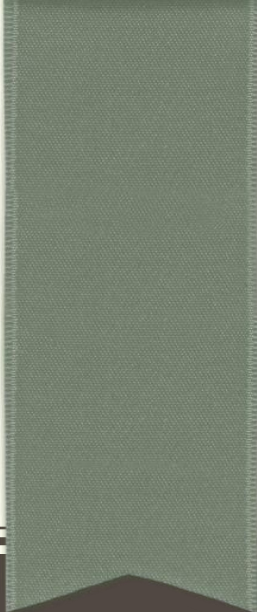
КЕРУВАННЯ КОРИСТУВАЦЬКИМИ ДАНИМИ ТА СТАНОМ ІНТЕРФЕЙСУ

Тема 05

Web-технології та web-дизайн

План лекції

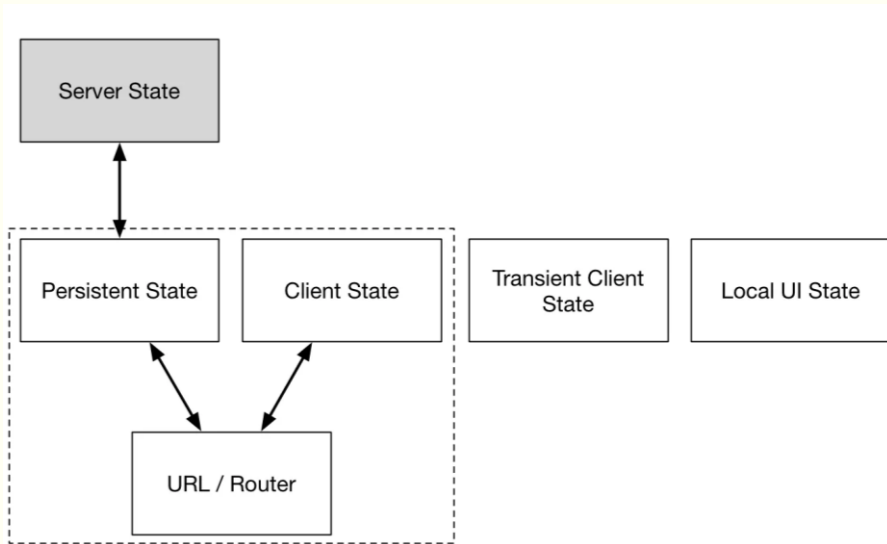
- 1) **Управління станом додатків у екосистемі JavaScript.**
- 2) **Інструменти React для керування станом.**
- 3) **JavaScript-бібліотеки для розширеного керування станом.**



УПРАВЛІННЯ СТАНОМ ДОДАТКІВ У ЕКОСИСТЕМІ JAVASCRIPT

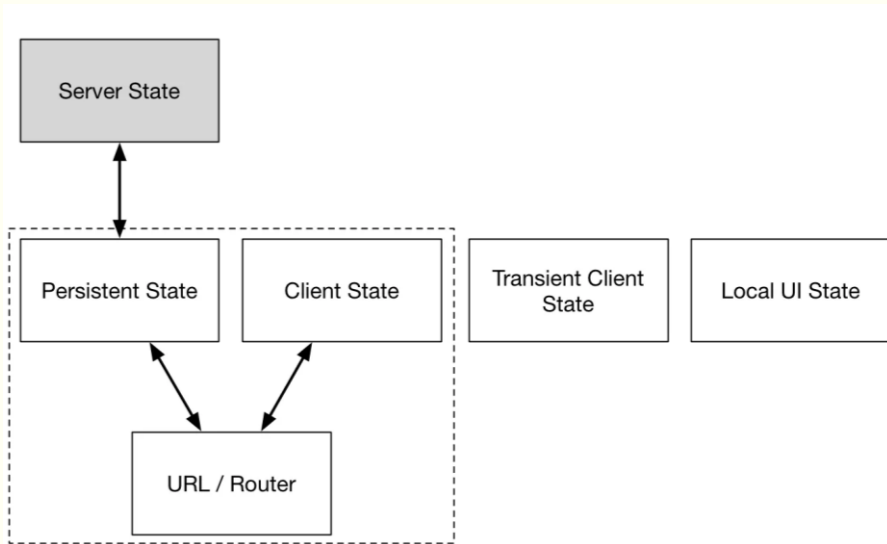
Питання 5.1.

Керування станом (у контексті React)



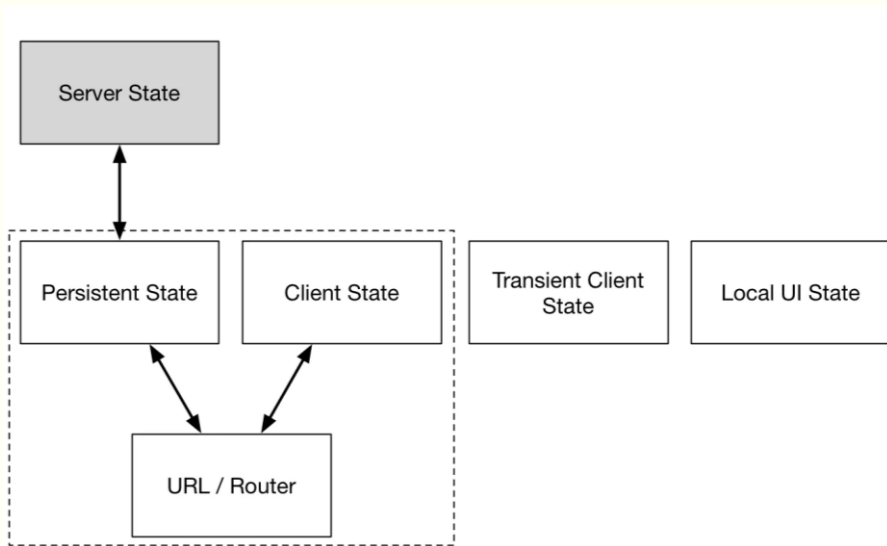
- Під час створення застосунку, який взаємодіє із сервером, ви часто працюєте навколо циклу запит/відповідь.
 - З часом занадто часті запити можуть зробити вашу програму повільною або марнотратною, тому поширений підхід – зберігати або кешувати вже отримані дані, особливо якщо вони змінюються не надто часто.
- На фронтенді більшість сучасних застосунків складаються з багатьох невеликих компонентів.
 - Кожен із цих компонентів має свій стан.
 - Кнопка може завантажуватися, бути активною або неактивною.
 - Форма може бути порожньою, частково заповненою або надісланою.
 - Деякий із цих станів є специфічним для компонента, тоді як деякий може бути спільним для всієї програми.
- Ми можемо згрупувати стани на два типи: *стан клієнта* та *стан сервера*.

Керування станом (у контексті React)



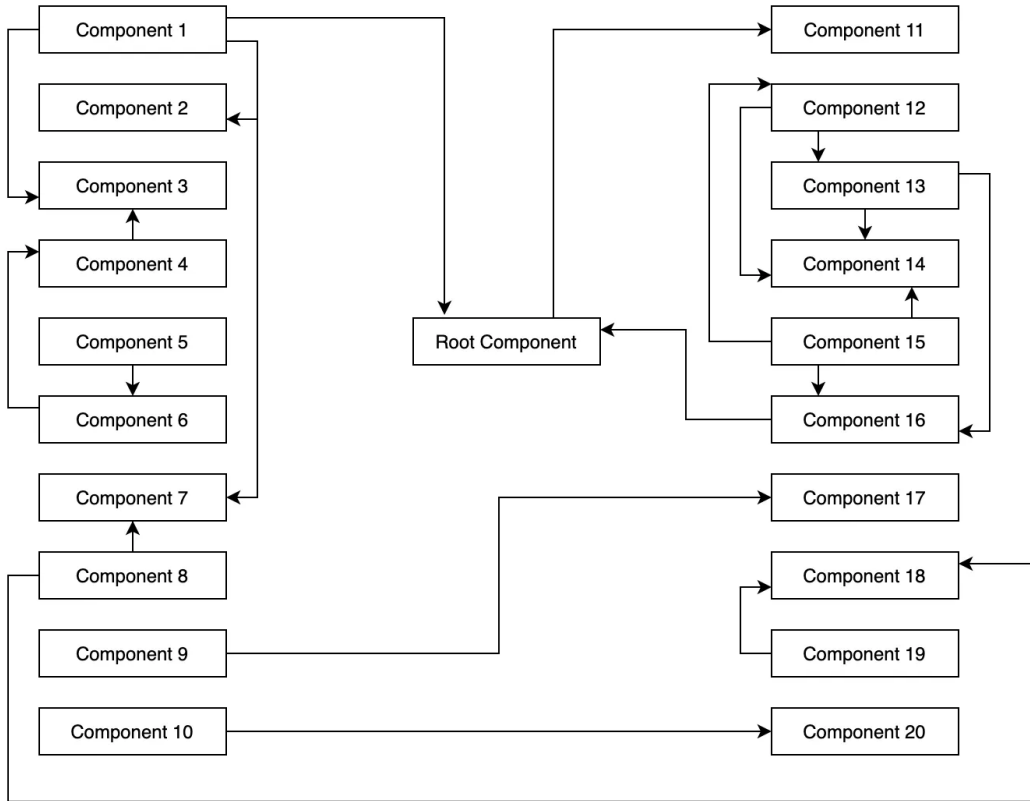
- *Стан сервера* можна вважати «постійним станом», оскільки дані, що зберігаються на сервері (зазвичай у БД), де видаляються лише за запитом.
 - Але збереження та отримання даних із сервера – це дорога операція, оскільки вони не зберігаються локально на обладнанні клієнта.
- З іншого боку, є дані, які зберігаються як *частина URL-адреси та стану маршрутизатора*.
 - Ці дані зазвичай використовуються для унікальної ідентифікації стану програми.
 - Рекомендується, щоб вони ніколи не містили конфіденційної інформації, такої як облікові дані користувача.
- *Стан клієнта* не зберігається на сервері та втрачається під час оновлення програми.
 - Цей тип стану тимчасово зберігає дії користувача в пам'яті.
 - Типовим прикладом є те, як YouTube запам'ятовує, де ви зупинили відео, тому наступного разу, коли повернетеся до того ж відео, воно почне відтворюватися з місця зупинки.
 - Але якщо ви оновите вебсайт YouTube, а потім відвідаєте те саме відео, воно почне відтворення з початку, оскільки цей стан зберігається лише до активного сеансу браузера.

Керування станом (у контексті React)



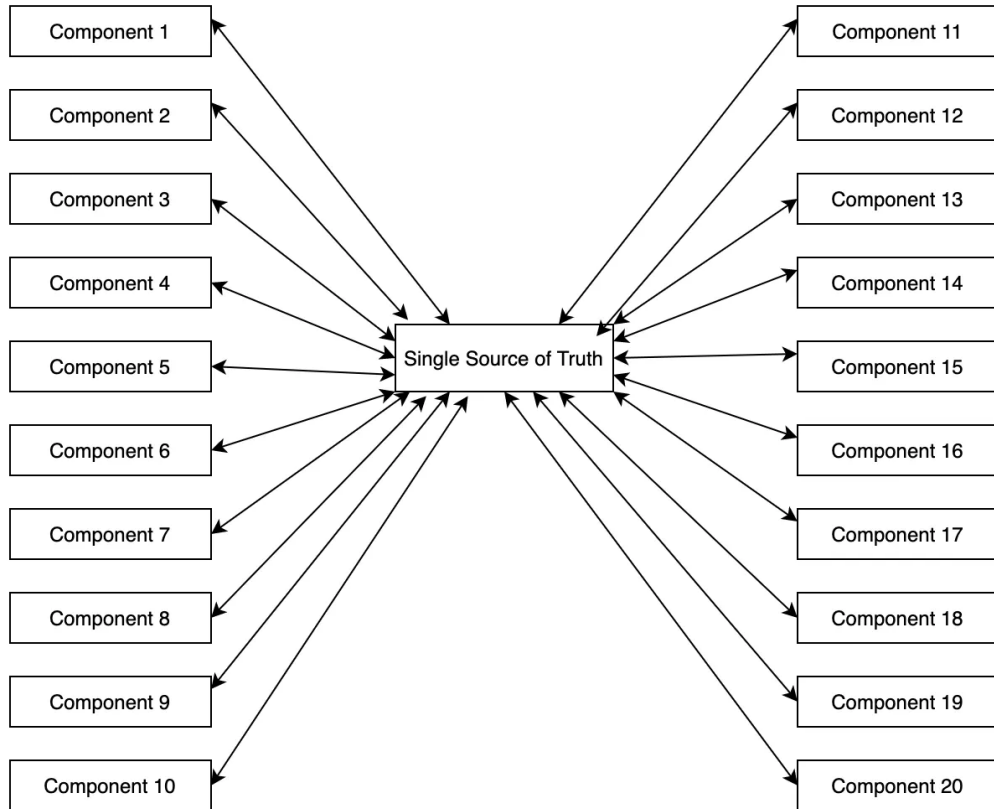
- Persistent State – це копія даних на стороні сервера, що зберігаються на клієнті.
 - Його також можна називати кешем стану сервера.
 - Клієнт працює з цими даними, щоб знати інформацію, що зберігається на сервері, та діяти відповідно.
- Тимчасовий (Transient) стан клієнта – це тимчасовий стан інтерфейсу користувача.
 - Він складається з усього стану інтерфейсу користувача, тобто повного стану фронтенд-застосунку.
 - Це допомагає у прийнятті рішень щодо модулів, компонентів та представлень для завантаження та показу користувачеві.
- Локальний стан інтерфейсу користувача (UI) стосується стану, локального для кожного компонента.
 - У сучасних клієнтських JavaScript-фреймворках кожен компонент має свій власний стан, який він може показувати або не показувати зовнішній області видимості.
 - Обмін даними зі стану компонента є дуже поширеним явищем, оскільки компоненти діляться внутрішніми станами для передачі інформації іншим компонентам.
 - На основі локального стану компонентів інтерфейсу користувача здійснюється маніпуляція представленням.

Керування станом (у контексті React)



- Обмін даними не є великою проблемою, коли ми маємо справу з малим застосунком, який містить приблизно 10–15 компонентів.
 - Типовий застосунок корпоративного рівня може мати сотні компонентів, і кожен компонент обмінюється даними з іншими компонентами.
 - На діаграмі компоненти спільно використовують дані відповідно до своєї «зручності».
 - Новачкам у фронтенді може здатись такий підхід зручним, оскільки надзвичайно легко передавати дані разом з іншим компонентом, який ви можете використовувати всередині свого компонента.
 - Але, як тільки ви закінчите та відійдете від цієї функціональності, важко відстежувати та запам'ятовувати потік даних.
 - Командна робота над репозиторієм ще більше ускладнює відстеження потоку даних та його розуміння.

Керування станом (у контексті React)



- Уявімо, що розробники вищезгаданої програми вирішать рефакторинг коду.

- Можемо побачити переваги використання шаблону проектування керування станами, оскільки дані рухаються лише в одному напрямку, а зворотна стрілка означає, що всі компоненти «підписуються» на «єдине джерело істини», яким зазвичай є глобальний об'єкт JavaScript.
- Популярний шаблон проектування для забезпечення цього називається **Flux**.
- Сучасні фреймворки JavaScript, такі як React, Angular та Vue, реалізують принципи проектування Flux, використовуючи **Redux** для React, **Ngrx** для Angular та **Vuex** для Vue.
- Flux робить стан передбачуваним. За домовленістю, кожна подія, яка запускається з компонента до центрального сховища даних, має додані метадані, які дозволяють інструментам браузера знати, звідки походить ця подія.
- Flux також робить код більш зручним у підтримці, оскільки застосовує єдиний принцип організації коду.
- Оскільки потік даних/подій є односпрямованим, набагато легше налагоджувати проблеми та звужувати джерело проблеми.

Оскільки центральне джерело даних – один об'єкт JavaScript, набагато простіше зберігати його в локальному сховищі браузера, сховищі сеансу або ж на сервері, щоб стан інтерфейсу користувача зберігався навіть після оновлення програми.

Управління станом сеансів (session)

Web Storage Options



LocalStorage

Small, persistent settings storage.



SessionStorage

Temporary tab/session data storage.



IndexedDB

Large/structured datasets or offline apps.



Cookies

Server needs access to the stored data.

- Керування сеансами відповідає за зберігання та підтримку даних користувача під час кількох запитів або взаємодій із вебзастосунком.
 - «Сеанс» означає період, протягом якого користувач активно взаємодіє з додатком, починаючи з моменту входу користувача в систему та закінчуючи виходом із системи або після певного періоду бездіяльності.
- Типи управління сесіями:
 - **Файли cookie:** невеликі фрагменти даних, що зберігаються у браузері користувача.
 - **Локальне сховище/сховище сеансів:** методи вебсховища для зберігання даних у браузері користувача.
 - **JWT (JSON Web Tokens):** безстановий спосіб обробки сесій за допомогою закодованих токенів.
 - **Сховища сеансів на стороні сервера:** постійне сховище сесій на сервері, яке відстежує сеанси користувачів.

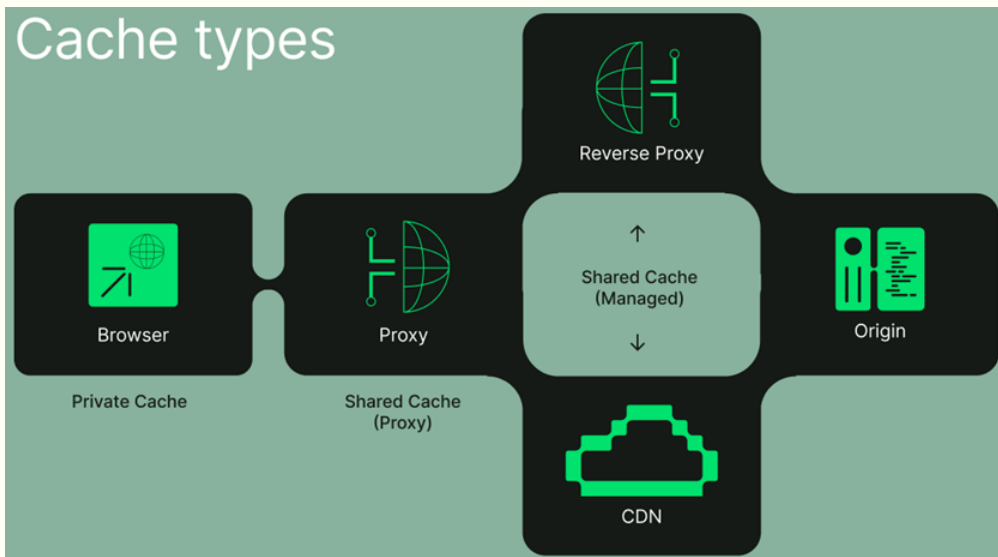
API вебсховища



CRITERIA	COOKIES	LOCAL STORAGE	SESSION STORAGE
MAXIMUM DATA SIZE	4 KB	5 MB	5 MB
BLOCKABLE BY USERS	YES	YES	YES
AUTO-EXPIRY OPTION	YES	NO	YES
SUPPORTED DATA TYPES	STRING ONLY	STRING ONLY	STRING ONLY
BROWSER SUPPORT	VERY HIGH	VERY HIGH	VERY HIGH
ACCESSIBLE SERVER SIDE	YES	NO	NO
DATA TRANSFERRED ON EVERY HTTP REQUEST	YES	NO	NO
EDITABLE BY USERS	YES	YES	YES
SUPPORTED ON SSL	YES	N/A	N/A
CAN BE ACCESSED ON	SERVER & CLIENT SIDE	CLIENT SIDE ONLY	CLIENT SIDE ONLY
CLEARING/DELETING	PHP, JS, AUTOMATIC	JS ONLY	JS & AUTOMATIC
LIFETIME	AS SPECIFIED	TILL DELETED	TILL TAB IS CLOSED
SECURE DATA STORAGE	NO	NO	NO

- Разом із сучасним вебом ми отримали нові API вебсховища (localStorage та sessionStorage) для зберігання на стороні клієнта, які дозволяють браузерам зберігати дані на стороні клієнта у вигляді пар ключ-значення.
- **LocalStorage** – це простий механізм зберігання пар «ключ-значення», що надається браузером.
 - Дані зберігаються **постійно**, тобто вони залишаються навіть після закриття та повторного відкриття браузера.
 - Найчастіше використовується для збереження налаштувань користувача (тема, макет); кешування невеликих відповідей API та зберігання стану легкої програми (наприклад, останньої відвіданої вкладки).
- **SessionStorage** працює подібно до LocalStorage, але прив'язаний до сеансу браузера.
 - Дані очищуються після закриття вкладки.
 - Найчастіше використовується для зберігання тимчасових даних форми, запобігання багаторазовому поданню та в ситуаціях, коли дані потрібні лише при відкритій сторінці.
 - Найкраще підходить для короткочасних даних, що зберігаються лише на одній вкладці.

Типи кешів



Вебкеш може бути спільним або приватним залежно від місця, де він знаходиться.

Кеш браузера є найпоширенішим місцем для кешування, і браузери зазвичай резервують для нього певне місце.

- Кеш браузера обмежений лише одним користувачем і, на відміну від інших кешів, він може зберігати «приватні» відповіді.

Проксі-кеші можуть обслуговувати сотні різних користувачів, які отримують доступ до одного й того ж контенту.

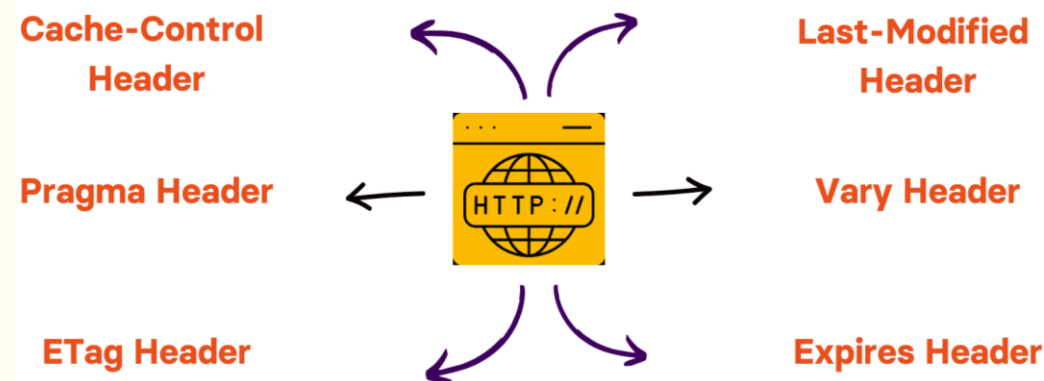
- Зазвичай вони впроваджуються на ширшому рівні, наприклад, інтернет-провайдерами або будь-якими іншими незалежними організаціями.

- Зворотний проксі-кеш (сурогатний кеш) впроваджується поблизу вихідних серверів, щоб зменшити навантаження на сервер.*

- На відміну від проксі-кешів від інтернет-провайдерів для зменшення використання пропускної здатності в мережі, сурогатні кеші впроваджуються поблизу вихідних серверів адміністраторами серверів, щоб зменшити навантаження на сервер.
- Хоча ви можете контролювати кеші зворотного проксі-сервера, ви *не можете уникнути або контролювати кеші браузера та проксі-сервера.*

Кешування заголовків

Key Types of HTTP Cache Headers



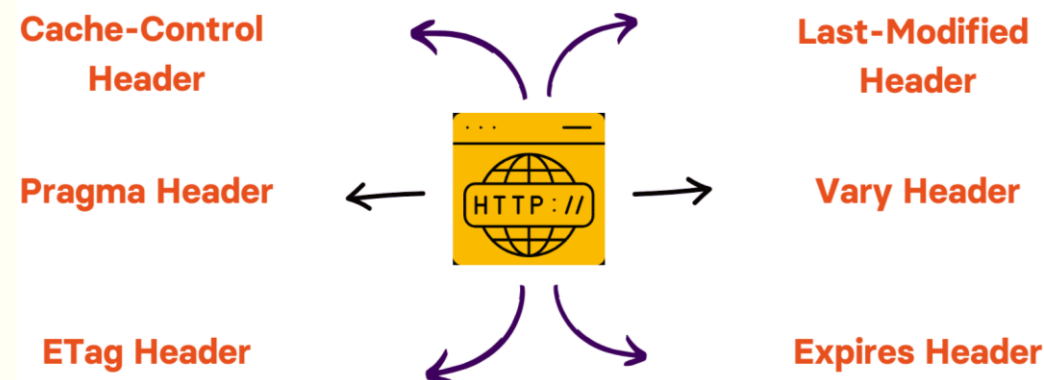
Expires: Mon, 13 Mar 2017 12:22:00 GMT

Постачальник контенту має переконатися, що він повертає правильні HTTP-заголовки, щоб примусово вказати кешам, як кешувати контент.

- **Expires.** Можливим значенням цього заголовка є абсолютна дата закінчення терміну дії; де дата має бути в GMT.
 - Дата не може бути старшою за рік, і якщо формат дати неправильний, контент вважатиметься застарілим.
 - Годинник у кеші має бути синхронізований з годинником на сервері, інакше бажаних результатів може не бути досягнуто.
 - Хоча заголовок Expires все ще дійсний і широко підтримується кешами, перевагу слід віддавати його наступнику HTTP/1.1, тобто Cache-Control.
- **Pragma.** Ще один старий метод, який нині замінено на Cache-Control.
 - Pragma: no-cache використовується тут і там в надії зупинити кешування відповіді.
 - Це не обов'язково спрацює; оскільки специфікація HTTP обговорює це в заголовках запиту, а в заголовках відповіді про це не згадується.
 - Натомість для керування кешуванням слід використовувати заголовок Cache-Control.

Кешування заголовків

Key Types of HTTP Cache Headers

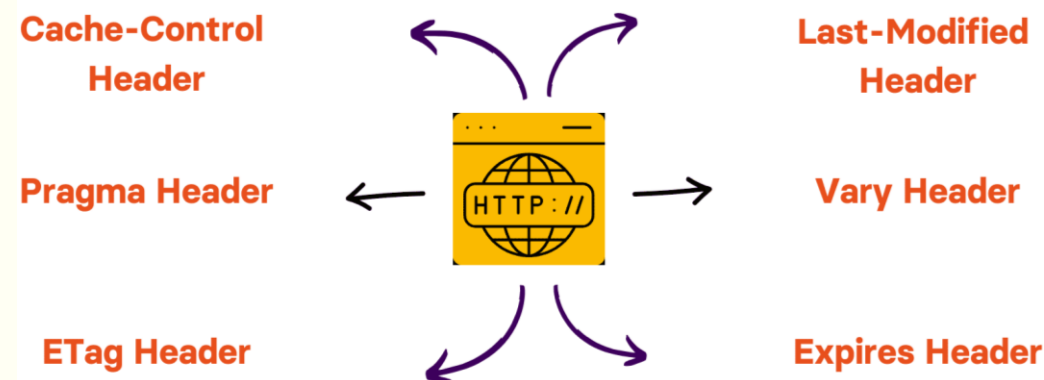


Cache-Control визначає, як довго і яким чином слід кешувати контент.

- Значення заголовка Cache-Control є складеним, тобто воно може мати кілька директив/значень.
- **Cache-Control: private** – контент не буде кешуватися в жодному з проксі-серверів, а буде кешуватися лише клієнтом (браузером).
 - Встановлення цього заголовка не забезпечить безпеку ваших даних; для цього вам все одно доведеться використовувати SSL.
- **Cache-Control: public** – окрім кешування клієнтом, контент також можуть кешувати проксі-сервери; обслуговуючи багатьох інших користувачів.
- **Cache-Control: no-store** – вміст не буде кешуватися жодним із кешів.
- **Cache-Control: max-age=3600, no-cache, public.**
 - *no-cache* вказує на те, що кеш можна підтримувати, але кешований вміст має бути повторно перевірений сервером (наприклад, за допомогою ETag) перед його відправкою.
 - Запит до сервера все ще надсилається, але для перевірки, а не для завантаження кешованого вмісту.
 - *max-age* вказує кількість секунд, протягом яких вміст буде кешуватися.
- **Cache-Control: s-maxage=3600, public** – префікс s- означає спільний доступ. Ця директива спеціально призначена для спільних кешів.
 - Якщо вона присутня, то перевизначатиме заголовки max-age та expires для спільного кешування.

Кешування заголовків

Key Types of HTTP Cache Headers

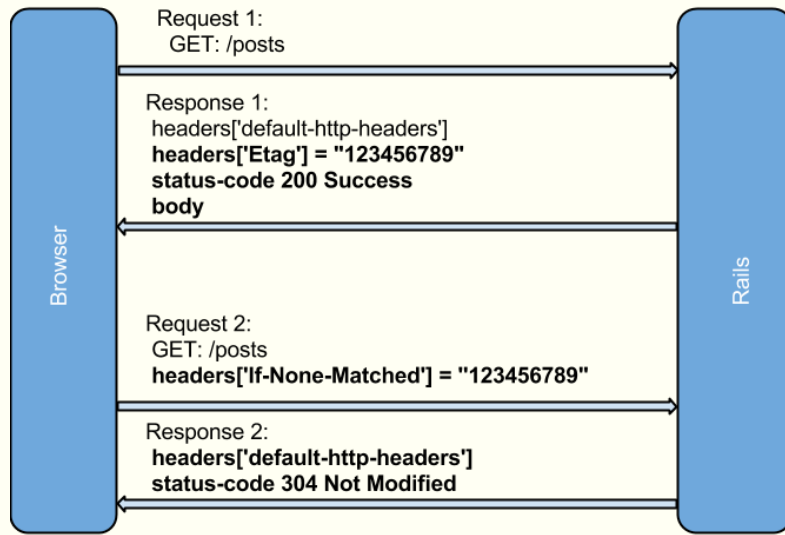
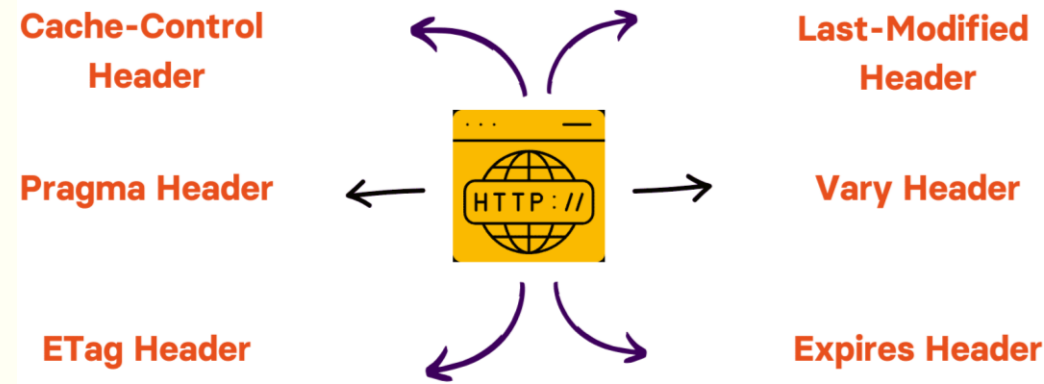


Cache-Control визначає, як довго і яким чином слід кешувати контент.

- **Cache-Control: max-age=3600, public, must-revalidate.**
 - Іноді може трапитися так, що якщо у вас виникли проблеми з мережею, і контент неможливо отримати з сервера, браузер може відображати застарілий контент без перевірки.
 - *must-revalidate* запобігає цьому. Якщо ця директива присутня, це означає, що застарілий контент не може бути відправлений у будь-якому випадку, і дані повинні бути повторно перевірені з сервера перед відправкою.
- **proxy-revalidate** схожий на *must-revalidate*, але визначає те ж для спільних або проксі-кешів.
- Ви можете комбінувати ці директиви по-різному для досягнення різних моделей кешування, проте *no-cache/no-store* та *public/private* є взаємовиключними.
- Якщо вказати обидва параметри: *no-store* та *no-cache*, параметр *no-store* матиме пріоритет над параметром *no-cache*.
- Для приватних/публічних запитів кеш неавтентифікованих запитів вважається публічним, а для автентифікованих – приватним.

Валідатори

Key Types of HTTP Cache Headers



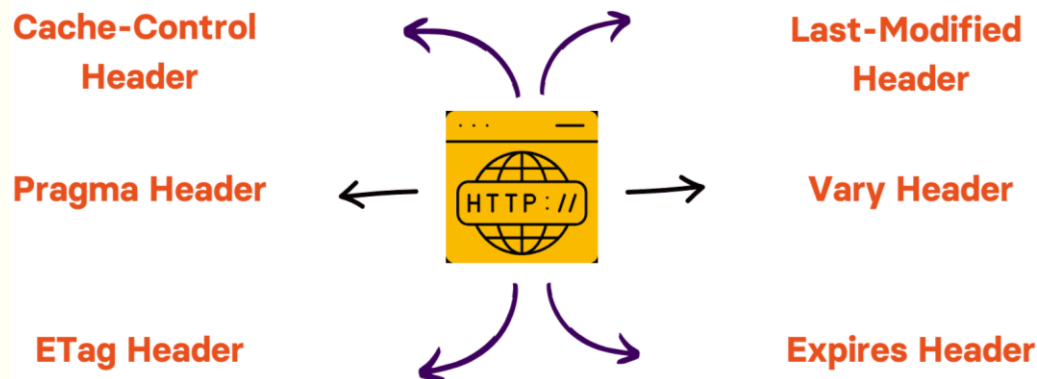
Досі ми не розглядали, як клієнт виконує перевірку з боку сервера.

Etag або «тег сутності» було введено у специфікаціях HTTP/1.1.

- Це унікальний ідентифікатор, який сервер пов'язує з певним ресурсом.
- Пізніше він використовується клієнтом для створення умовних HTTP-запитів типу «надайте мені цей ресурс, якщо ETag не збігається з ETag, який у мене є», і контент завантажується лише тоді, коли etag-и не збігаються.
- Метод, за допомогою якого генерується ETag, не вказано в документації HTTP – зазвичай для призначення etag кожній версії ресурсу використовується якась хеш-функція, стійка до колізій.
- Може бути два типи etag: сильні та слабкі.
- ETag: "j82j8232ha7sdh0q2882" – Strong Etag
- ETag: W/"j82j8232ha7sdh0q2882" – Weak Etag
- *Сильний перевірочний ETag* означає, що два ресурси абсолютно однакові, і між ними немає жодної різниці.
- *Слабкий ETag* означає, що два ресурси, хоча й не є абсолютно однаковими, можна вважати однаковими. Слабкі etags можуть бути корисними, наприклад, для динамічного контенту.

Валідатори

Key Types of HTTP Cache Headers



```
HTTP/1.1 200
accept-ranges: bytes
cache-control: max-age=1734
content-length: 235763
content-type: application/octet-stream
date: Wed, 06 Nov 2019 05:07:44 GMT
etag: "2eac9604a0b5ce400a0630f2d9fdb30b"
last-modified: Fri, 25 Oct 2019 04:48:51 GMT
server: AmazonS3
server-timing: cdn-cache; desc=HIT
server-timing: edge; dur=3
status: 304
x-amz-id-2: znxTgJHH75/A4AgRwB66zjZPrhDDd9yRcUOU2N50jfyZ6fpbG/DW/Pbe28x+dTx51Dx+ewQ/BO4=
x-amz-request-id: 141842A51C2EE9D8
```

Сервер може містити заголовок **Last-Modified**, який вказує дату та час останньої зміни певного вмісту.

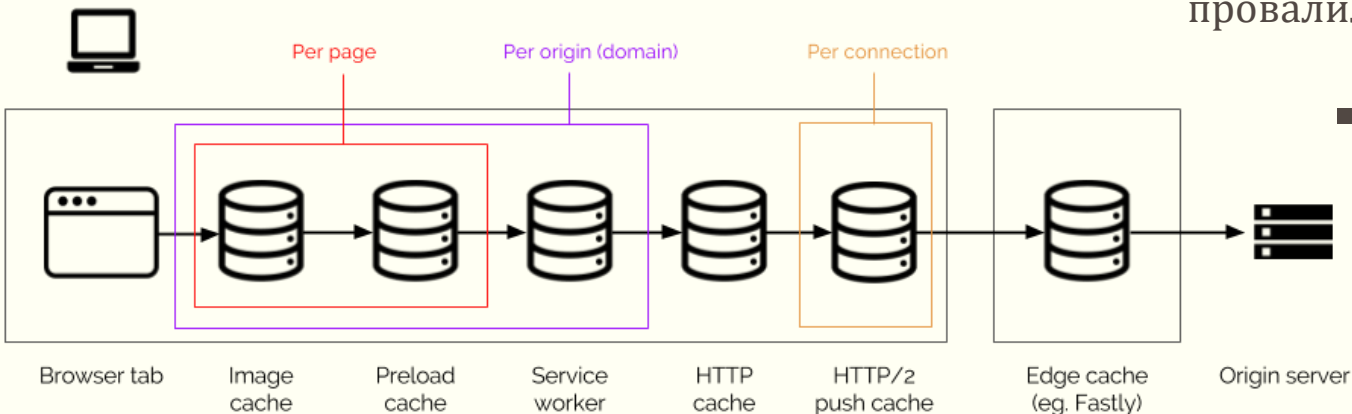
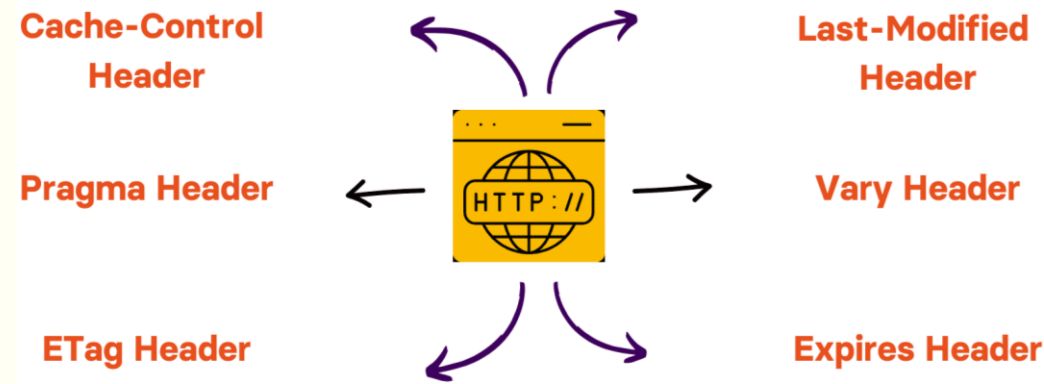
- Коли контент стає застарілим, клієнт надсилає умовний запит до сервера, включаючи дату останньої зміни, яка є в заголовку If-Modified-Since, щоб отримати оновлену дату останньої зміни.
- Якщо вона збігається з датою, встановленою клієнтом, дата останньої зміни контенту оновлюється, щоб вважатися свіжою ще протягом n секунд.
- Якщо отримана дата останньої зміни не збігається з датою, встановленою клієнтом, контент перезавантажується із сервера та замінюється контентом, встановленою клієнтом.

що робити, якщо кешований контент має призначені як Last-Modified, так і ETag?

- Мають використовуватися обидва, тобто повторного завантаження ресурсу не буде тоді і тільки тоді, коли ETag збігається з щойно отриманим, а також з датою останньої зміни.
- Якщо ETag не збігається або дата останньої зміни більша за дату з сервера, контент потрібно завантажити знову.

Заголовок Vary

Key Types of HTTP Cache Headers



- традиційне використання Vary полягає в узгодженні контенту за допомогою заголовків Асерт, Асерт-Language та Асерт-Encoding.

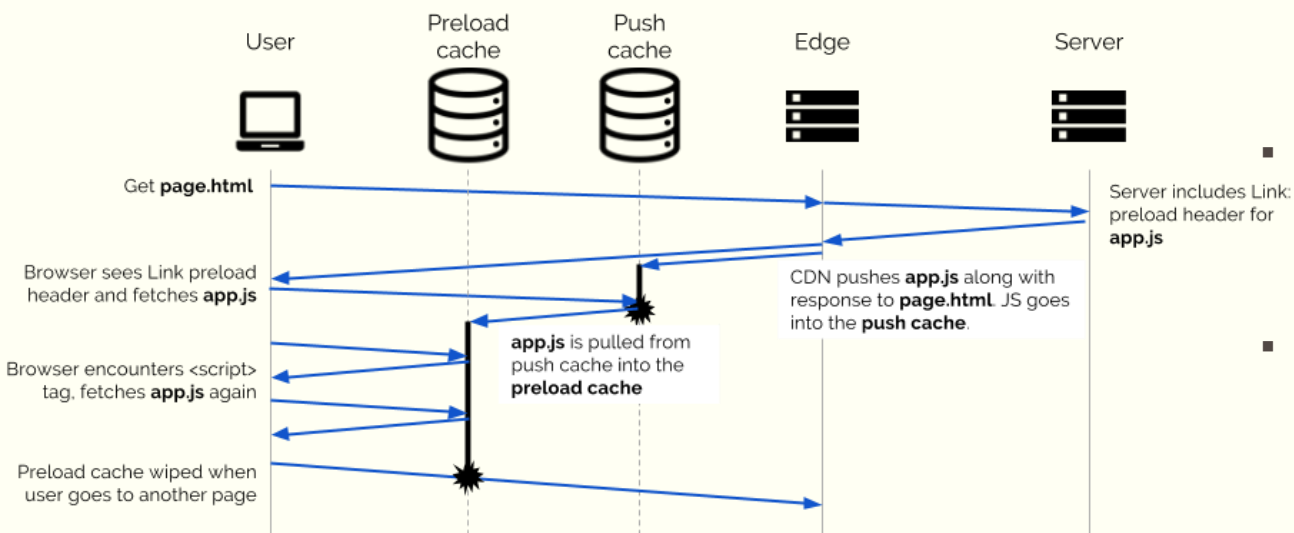
- Оскільки одна URL-адреса може генерувати різні відповіді залежно від значення заголовка Асерт, будь-який кеш, який зберігає цю відповідь, повинен знати, що цей заголовок важливий.
- Сервер так повідомляє нам, що заголовок Асерт важливий:

Vary: Accept

- Так зване «узгодження контенту» було чудовою ідеєю, але вона провалилася.

- Якщо заголовок не працює для узгодження контенту, чому він досі такий популярний, і як браузери з цим справляються?
 - Багато різних кешів для різних, специфічних цілей!

Заголовок Vary

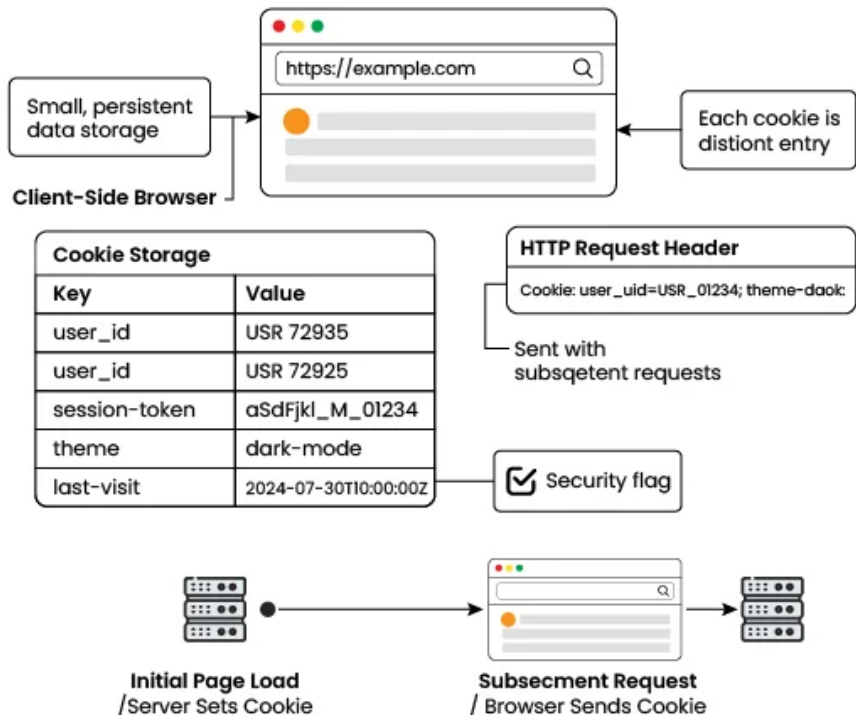


Детальніше про заголовок Vary

- **Кеш зображень** обмежений сторінкою, який зберігає декодовані дані зображень, тож, наприклад, якщо ви включаєте одне й те ж зображення на сторінку кілька разів, браузеру потрібно буде завантажити та декодувати його лише один раз.
- **Кеш попереднього завантаження** також має обсяг сторінки та зберігає все, що було попередньо завантажено, у заголовок Link або тегу `<link rel="preload">`, навіть якщо ресурс зазвичай не кешується. Як і кеш зображень, кеш попереднього завантаження знищується, коли користувач залишає сторінку.
- **API кешу сервісного воркера** забезпечує серверну частину кешу з програмованим інтерфейсом; тому тут нічого не зберігається, якщо ви спеціально не розмістите це туди за допомогою коду JavaScript у сервісному воркері.
 - Також воно буде перевірено лише в тому випадку, якщо ви явно зробите це в `fetch`-обробнику сервісного воркера.
 - Кеш сервісного воркера має область походження, і, хоча його постійність не гарантується, він є більш постійним, ніж HTTP-кеш браузера.
- **HTTP-кеш** – основний кеш, єдиний кеш, який звертає увагу на заголовки кешу рівня HTTP, такі як `Cache-Control`, і поєднує їх з власними евристичними правилами браузера, щоб визначити, чи кешувати щось і на який термін.
 - Якщо два непов'язані веб-сайти завантажують один і той самий ресурс (наприклад, Google Analytics), вони можуть мати спільний доступ до одного кешу.
- **Кеш HTTP/2 push** знаходиться разом із з'єднанням і зберігає об'єкти, які були надіслані із сервера, але ще не були запитувані жодною сторінкою, що використовує це з'єднання.
 - Він обмежений сторінками, що використовують певне з'єднання, що по суті те ж, що й обмежений одним джерелом, але він також знищується, коли з'єднання закривається.

HTTP-cookies у браузері

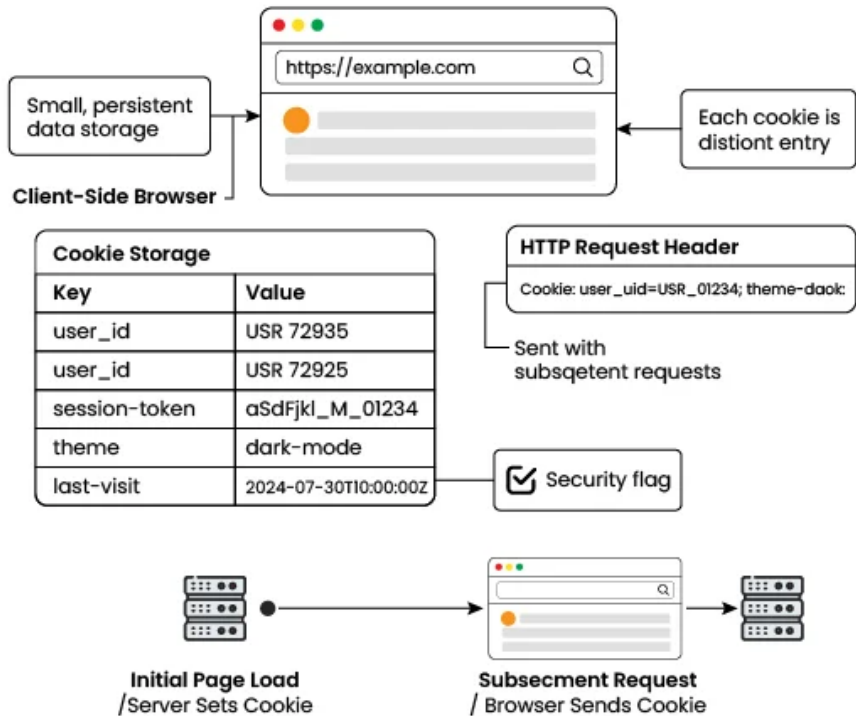
JavaScript cookies



- Файл cookie – це один або декілька фрагментів інформації, що зберігаються у вигляді текстових рядків на вашому комп'ютері.
 - cookie надсилається вашому браузеру веб-сервером через HTTP-запит.
 - Потім браузер повертає файл cookie на сервер під час наступного посилання на сторінку.
- Спеціальні файли cookie використовуються для ідентифікації конкретних користувачів та покращення їхнього досвіду перегляду вебсторінок.
 - Дані, що зберігаються в файлах cookie, створюються сервером під час вашого підключення.
 - Ці дані позначені унікальним ідентифікатором для вас та вашого комп'ютера.
 - Коли файли cookie обмінюються між вашим комп'ютером і мережевим сервером, сервер зчитує ідентифікатор і знає, яку інформацію саме вам надавати.

HTTP-cookies у браузері

JavaScript cookies



- На стороні клієнта ми можемо отримати доступ до файлів cookie через властивість документа `cookie`.
`document.cookie="country=Ukraine"`
- За замовчуванням час дії файлу cookie буде визначатися як час сеансу (*сеансові cookie-файли*).
 - Якщо поточний сеанс перегляду завершиться, термін дії файлу cookie втрачає чинність.
 - Але ми також можемо встановити час закінчення терміну дії для файлів cookie за допомогою параметрів `expires` або `max-age` (*постійні cookie-файли*).

```
// persistent cookies

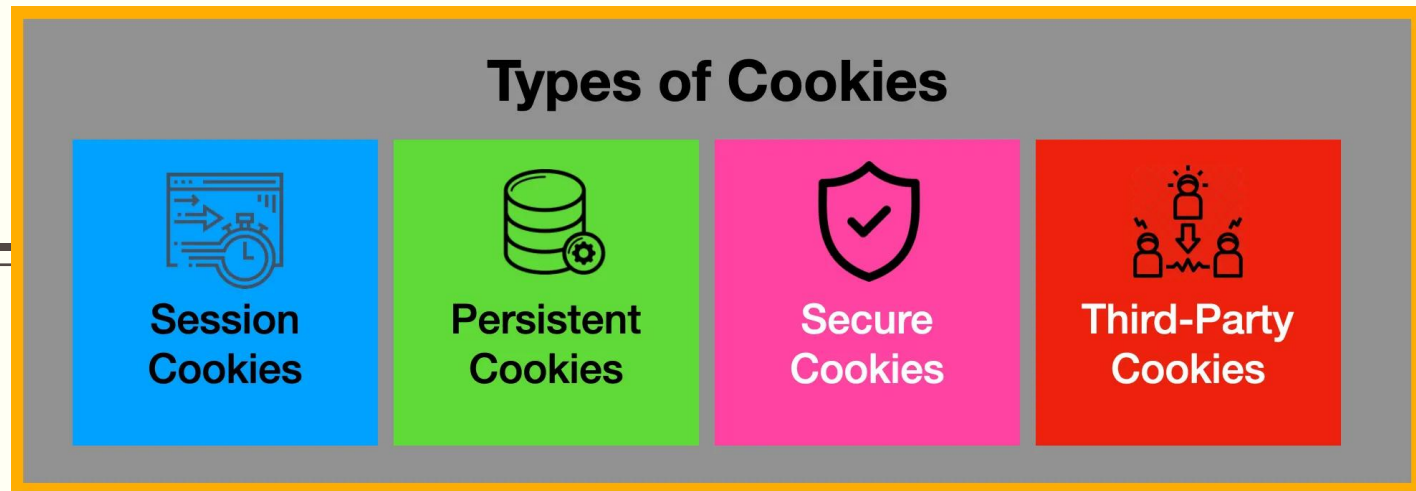
var expires = (new Date(Date.now()+ 86400*1000)). toUTCString();

document.cookie = "cookieName=cookieValue; expires=" + expires + ";path=/;"

document.cookie = "cookieName=cookieValue; max-age=86400; path=/;"
```

- Файл cookie з від'ємним або нульовим віком негайно втрачає чинність.

HTTP-cookies у браузері.



- **Сесійні файли cookie** (ідентифікатори сеансу, токени сеансу) – це тип файлів cookie для зберігання інформації про сеанс користувача.
 - Це тимчасові файли cookie, які створюються, коли користувач відвідує вебсайт, і зазвичай видаляються, коли користувач закриває веб-браузер.
 - Основна мета сесійних файлів cookie – унікально ідентифікувати сеанс користувача та зберігати інформацію, пов'язану з цим сеансом.
- **Постійні файли cookie** – це тип файлів cookie, що використовуються для зберігання інформації на пристрої користувача протягом тривалого періоду.
 - На відміну від сесійних файлів cookie, які є тимчасовими та термін дії яких закінчується, коли користувач закриває браузер, постійні файли cookie зберігаються на пристрої користувача протягом усіх сеансів браузера.
 - Вони мають певний термін дії та часто використовуються для довгострокового відстеження, автентифікації користувачів та персоналізації.

HTTP-cookies у браузері.



- **Безпечні файли cookie** – це тип HTTP-файлів cookie, які передаються лише через безпечні, зашифровані з'єднання.
 - Вони забезпечують додатковий рівень безпеки, гарантуючи передачу даних файлів cookie між вебсервером і клієнтом через з'єднання HTTPS (HTTP Secure).
 - Безпечні файли cookie допомагають захистити конфіденційну інформацію від перехоплення або підробки під час передачі.
- **Сторонні файли cookie** – це файли cookie, які встановлюються доменами, відмінними від того, який користувач наразі відвідує.
 - Вони створюються зовнішніми вебсайтами або доменами для відстеження активності користувачів, збору аналітичних даних та показу персоналізованого контенту чи реклами на різних вебсайтах.
 - Сторонні файли cookie дозволяють рекламодавцям та аналітичним службам збирати інформацію про поведінку користувача під час перегляду на кількох сайтах.
 - Коли користувач переглядає вебсайт для мобільних телефонів, рекомендації щодо мобільних телефонів починають надходити, коли користувач відвідує інші вебсайти. Все це завдяки стороннім файлам cookie.

HTTP-cookies у браузері



- На стороні сервера ми можемо отримати доступ до файлів cookie, читати файли cookie та змінювати їх за допомогою заголовка відповіді `Set-Cookie` та файлу cookie в заголовку запиту.
 - Коли сервер відповідає на HTTP-запит клієнта, запитуйте також HTTP-заголовки, що описують, як сервер обробляв цей запит. Відповідь може містити будь-яку кількість заголовків `Set-Cookie`, по одному для кожного файлу cookie, який потрібно встановити.
 - Також сервер може зчитувати файл cookie, переданий через http-виклики, у заголовку запиту cookie.

```
HTTP/2 200

Content-Type: text/html; charset=UTF-8

Set-Cookie: nameA=valueA; Domain=example.com; HTTPOnly

Set-Cookie: nameB=valueB; Domain=example.com; Max-Age=10

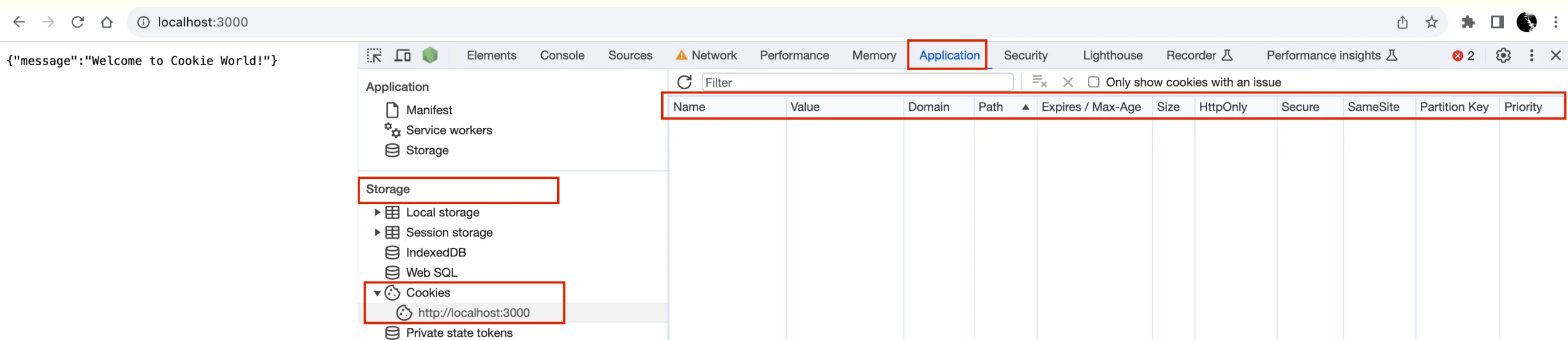
Set-Cookie: nameC=valueC; Domain=example.com; Secure
```

```
var cookies = request.headers.cookie;

console.log(cookies);

// will give you cookies passed from client side to server
// side in the request header.
```

Де можна побачити файли cookie у браузері?



- Один файл cookie може зберігати багато метаданих: назва файлу cookie (ключ), значення (value), домен, шлях, дата закінчення терміну дії, розмір, HttpOnly-кукі (bool), безпечність (bool), параметр SameSite (bool), ключ розділу та пріоритет (специфічний для Chrome).

```
username=JohnDoe; expires=Fri, 31 Dec 2024 12:00:00 UTC; path=/;
```

HTTP-cookies у браузері. Атрибути Cookie

- **Атрибут Domain** повідомляє браузеру, яким хостам дозволено доступ до файлу cookie.
 - Якщо його не вказано, за замовчуванням використовується той самий хост, який встановив файл cookie.
 - Під час доступу до файлу cookie за допомогою клієнтського JavaScript доступні лише ті файли cookie, які мають той самий домен, що й у рядку URL-адреси.
- **Атрибут Path** вказує шлях в URL-адресі запиту, який має бути присутнім для доступу до файлу cookie.
- **Атрибут Expires** встановлює термін дії, після якого файли cookie знищуються.
 - Це може стати в нагоді, коли ви використовуєте файл cookie для перевірки, чи бачив користувач міжсторінкову рекламу; ви можете встановити термін дії файлу cookie через місяць, щоб реклама могла відображатися знову через місяць.
- **Атрибут Secure.** Файл cookie з атрибутом Secure надсилає дані на сервер лише через захищений протокол HTTPS і ніколи не через протокол HTTP (за винятком випадків, коли сервер localhost).
- **Атрибут HttpOnly** дозволяє доступ до файлів cookie лише через сервер.
 - Лише сервер може встановлювати їх через заголовки відповіді. Якщо вони надсилаються на сервер із заголовками кожного наступного запиту, вони не будуть доступні через JavaScript на стороні клієнта.
 - Це може частково допомогти захистити файли cookie з конфіденційною інформацією.
 - Якщо для файлу cookie було встановлено прапорець http-only, ви не зможете прочитати його за допомогою JavaScript; це захід безпеки для запобігання перехопленню сеансу, і його слід встановити для будь-якого сурогатного ідентифікатора, включаючи файли cookie сеансу.
- **Атрибут SameSite** використовується для контролю надсилання файлу cookie в міжсайтових запитах (сайт В надсилає запит на сайт А).

Зразки роботи з cookies в Node.js та Express

localhost:3000/setCookies

Elements Console Sources Network Performance Memory Application Security Lighthouse Recorder Performance insights

Application

- Manifest
- Service workers
- Storage

Storage

- Local storage
- Session storage
- IndexedDB
- Web SQL
- Cookies
 - http://localhost:3000

Filter

Only show cookies with an issue

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Partition Key	Priority
userEmail	abc%40gmail.com	localhost	/	Session	24					Medium

```
import express from 'express';
import cookieParser from 'cookie-parser';

const host = process.env.HOST ?? 'localhost';
const port = process.env.PORT ? Number(process.env.PORT) : 3000;

const app = express();
app.use(cookieParser());

app.get('/', (req, res) => {
  res.send({ message: 'Welcome to Cookie World!' });
});
```

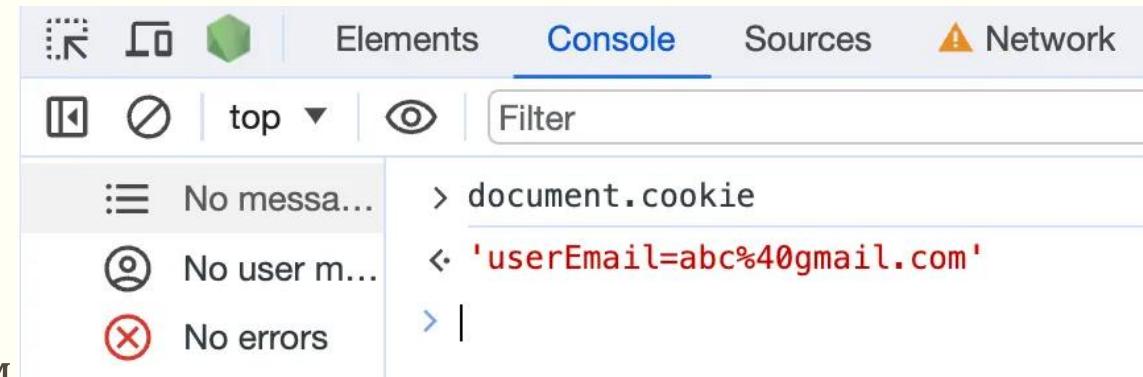
```
app.get('/setCookies', (req, res) => {
  res.cookie('userEmail', 'abc@gmail.com');
  res.send({ message: 'Cookie has been set!' });
});

app.get('/getCookies', (req, res) => {
  console.log(req.cookies);
  res.send(req.cookies);
});

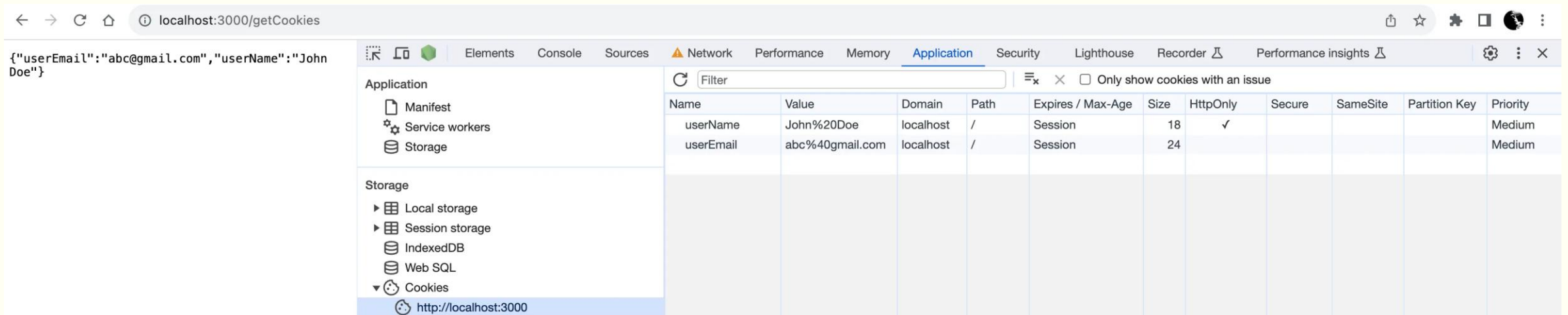
app.listen(port, host, () => {
  console.log(`[ ready ] http://${host}:${port}`);
});
```

Зразки роботи з cookies в Node.js та Express

```
app.get('/setCookies', (req, res) => {  
  res.cookie('userEmail', 'abc@gmail.com');  
  res.cookie('userName', 'John Doe', { httpOnly: true });  
  res.send({ message: 'Cookie has been set!' });  
});
```



- Вищезазначений файл cookie є незахищеним.
 - Розробник або хакер може отримати доступ до файлу cookie за допомогою цього document-об'єкта.
 - Це створює ризик маніпулювання файлом cookie або його видалення.
 - Якщо сервер встановлює конфігурацію об'єкта CookieOptions як { httpOnly: true }, то cookie не буде доступним для document-об'єкта.



Зразки роботи з cookies в Node.js та Express

```
app.get('/setCookies', (req, res) => {  
  res.cookie('userEmail', 'abc@gmail.com');  
  res.cookie('userName', 'John Doe', { httpOnly: true, maxAge: 60000 });  
  res.send({ message: 'Cookie has been set!' });  
});
```

- **maxAge** може бути використаний для встановлення дати або часу закінчення терміну дії.
 - Наприклад, термін дії файлу cookie має закінчитися через 60 секунд – надайте **maxAge: 60000**.
 - Щойно ми отримаємо файли cookie після 60 секунд, **userName** буде видалено.
 - Це часто можна використовувати для запуску автентифікації, скажімо, через 60 хвилин, якщо файл cookie відсутній.

localhost:3000/setCookies

["message":"Cookie has been set!"]

Application

- Manifest
- Service workers
- Storage

Storage

- Local storage
- Session storage

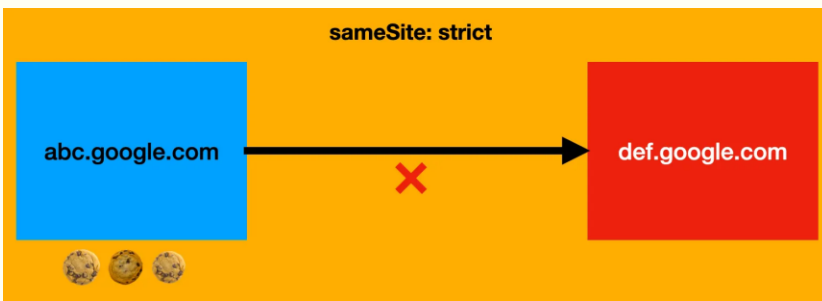
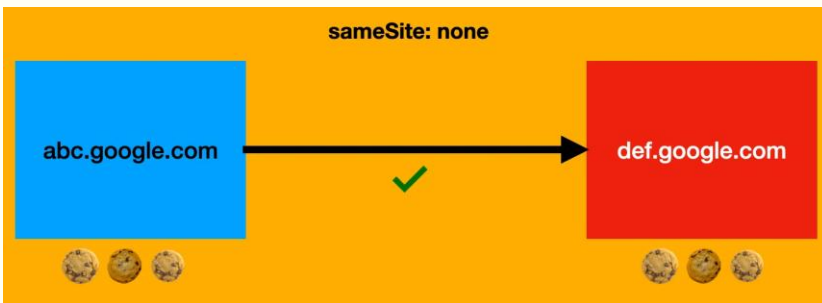
Filter

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	S..	SameSite	Partition Key	Priority
userName	John%20Doe	localhost	/	2023-12-06T12:16:36.904Z	18	✓				Medium
userEmail	abc%40gmail.com	localhost	/	Session	24					Medium

Зразки роботи з cookies в Node.js та Express

```
app.get('/setCookies', (req, res) => {  
  res.cookie('userEmail', 'abc@gmail.com');  
  res.cookie('userName', 'John Doe', {  
    httpOnly: true,  
    maxAge: 10000,  
    secure: true,  
  });  
  res.send({ message: 'Cookie has been set!' });  
});
```

- У наведеному коді файл cookie може бути надісланий із сервера для доступу лише через https-вебсайти, а не через http.
- Додатково може налаштовуватись атрибут **sameSite**:
 - **sameSite:none** – передає файли cookie з домену abc.google.com до домену def.google.com.
 - **sameSite:strict** – не передає файли cookie з домену abc.google.com до домену def.google.com
 - **sameSite:lax** – умовно передає файли cookie з домену abc.google.com до домену def.google.com. Файл cookie буде надіслано лише під час *навігації верхнього рівня* (наприклад, під час переходу за посиланням), *ініційованої тим же сайтом*.
 - Якщо навігація є навігацією верхнього рівня, ініційованою користувачем, наприклад, натисканням на посилання, файли cookie з sameSite=Lax вихідного домену (abc.google.com) будуть надіслані разом із запитом до цільового домену (def.google.com).



Основи роботи з сеансами

```
const express = require("express");
const app = express();
const port = 8000;

// Login middleware
const login = (req, res, next) => {
  if (req.params.id == 1234) {
    next();
  } else {
    res.send("User ID is not valid");
  }
};

// Protected route
app.get("/salary", (req, res) => {
  res.send("User salary is 20K");
});

// Login route
app.get("/login/:id", login, (req, res) => {
  res.send("This route is protected and login successful");
});

app.listen(port, () => {
  console.log(`Server running on port ${port}`);
});
```

- HTTP не зберігає стан – кожен запит є незалежним.
 - Це означає, що сервер не може запам'ятати, чи користувач увійшов у систему, якщо ми не використовуємо сесії.
 - Сесії дозволяють серверу «запам'ятовувати» користувачів між кількома запитами.
- Основна проблема поточного коду:
 - Користувач входить у систему через/login/:id
 - Якщо той самий користувач відвідує /salary, сервер не має доказів того, що користувач увійшов у систему
 - Вам потрібно буде повторити перевірку входу для кожного захищеного маршруту.

Основи роботи з сеансами

```
const express = require("express");
const session = require("express-session");
const app = express();
app.use(express.json());
const port = 8000;

app.use(session({
  secret: "mysecretkey",
  resave: false,
  saveUninitialized: false,
  cookie: { maxAge: 1000 * 60 * 5 } // 5 minutes
}));

// Login route
app.get("/login", (req, res) => {
  req.session.user = { name: "Atiful Haque", age: 23 };
  res.send("Successfully logged in");
});

// Protected route
app.get("/protected_route", (req, res) => {
  if (req.session.user) {
    res.send("You can access this route");
  } else {
    res.send("Please login first");
  }
});

app.listen(port, () => {
  console.log(`Server running on port ${port}`);
});
```

■ Концептуально сеанс працює так:

- 1) Користувач входить за допомогою імені/пароля.
- 2) Сервер перевіряє облікові дані та створює **сеанс**.
- 3) Сервер генерує **ідентифікатор сесії** (як ключ).
- 4) Сервер зберігає дані сеансу в пам'яті, базі даних або Redis:

```
sessionId: 83jd9f93jd9fj
user: { id: 10, name: "John" }
```

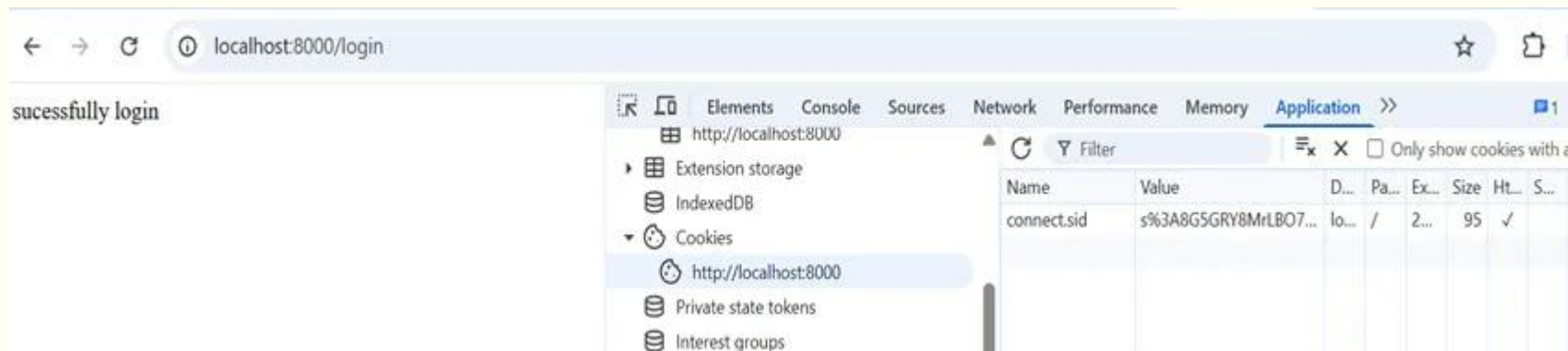
- 5) Сервер надсилає **файл cookie** до браузера:
- ```
connect.sid=83jd9f93jd9fj
```
- 6) На кожен запит браузер автоматично надсилає файл cookie.
  - 7) Сервер перевіряє ідентифікатор сесії у своєму сховищі, щоб дозволити або заборонити доступ.

## ■ Щойно маршрут /login відвідується, сервер перевіряє, чи вже є у користувача сеанс.

- Якщо сеансу не існує, сервер генерує новий ідентифікатор сеансу.
- Цей ідентифікатор сеансу потім надсилається назад до браузера як файл cookie у відповіді.

# Основи роботи з сеансами

- connect.sid – це фактичний сеансовий cookie, який надсилає сервер.



- якщо вивести `req.sessionID`, ви отримаєте текст, який присутній в останньому рядку зображення.

```
server is running in port 8000
Session {
 cookie: {
 path: '/',
 _expires: 2025-12-11T19:16:52.569Z,
 originalMaxAge: 300000,
 httpOnly: true
 },
 user: { user: 'atiful haque', age: 23 }
}
GA0g60l65Z_ppYw61MBtv8nMXkKAiqH2
```

# Основи роботи з сесансами

```
const crypto = require("crypto");
const sessions = {};

// Helper to set cookie
function setCookie(res, name, value, options = {}) {
 let cookie = `${name}=${value}`;
 if (options.maxAge) cookie += `; Max-Age=${options.maxAge}`;
 if (options.httpOnly) cookie += "; HttpOnly";
 res.setHeader("Set-Cookie", cookie);
}

// Login
app.get("/login", (req, res) => {
 const sessionId = crypto.randomBytes(16).toString("hex");
 sessions[sessionId] = { user: "Atiful Haque", age: 23 };
 setCookie(res, "my_session_id", sessionId, { maxAge: 300, httpOnly: true });
 res.send(`Logged in! Session ID: ${sessionId}`);
});

// Protected route
app.get("/protected", (req, res) => {
 const cookie = req.headers.cookie;
 if (!cookie) return res.send("Please login first");
 const match = cookie.match(/my_session_id=([^;]+)/);
 if (!match) return res.send("Please login first");

 const session = sessions[match[1]];
 if (!session) return res.send("Session expired or not found");

 res.send(`Hello ${session.user}, you can access this route`);
});
```

The user sends login request

The server authorizes the login, sends a session to the database, and returns a cookie containing the session ID to the user



User sends new request (with a cookie)



- Сесанс можна створити вручну, без бібліотек.
- Важливі атрибути сесансу:
  - **secret: "довільний рядок"** – секретний ключ, який використовується для підписання cookie ідентифікатора сесансу.
    - Коли ви входите в систему, сервер створює сесанс з ідентифікатором сесансу, наприклад, "abc".
    - Сервер використовує секретний ключ для створення підпису для цього ідентифікатора сесансу (`connect.sid = sessionId.signature`).
    - Під час наступних запитів сервер перевіряє cookie: він перераховує підпис, використовуючи секретний код.
    - Якщо збігається, сесанс дійсний; якщо ні, файл cookie відхиляється.

```

const express = require("express");
const session = require("express-session");

const app = express();

app.use(session({// Session configuration
 secret: "mysecretkey", // 1. Secret key to sign the cookie
 resave: false, // 2. Save session only if modified
 saveUninitialized: false, // 3. Save only when session data exists
 cookie: { // 4. Cookie settings
 maxAge: 1000 * 60 * 5, // Cookie expires in 5 minutes
 httpOnly: true, // Cookie cannot be accessed by client-side JS
 secure: false, // Cookie sent only over HTTPS (false for local testing)
 sameSite: "lax" // Controls when the cookie is sent (CSRF protection)
 }
}));

app.get("/login", (req, res) => {// Login route
 req.session.user = {
 name: "Atiful Haque", age: 23
 };
 res.send("Successfully logged in. Session created!");
});

app.get("/profile", (req, res) => {// Profile route (requires session)
 if (req.session.user) {
 res.send(`Welcome ${req.session.user.name}, age: ${req.session.user.age}`);
 } else {
 res.send("You are not logged in.");
 }
});

app.get("/logout", (req, res) => {// Logout route
 req.session.destroy(err => {
 if (err) { return res.send("Error logging out"); }

 res.send("Logged out successfully!");
 });
});

```

## Основи роботи з сеансами

- Важливі атрибути сеансу:
  - resave** – логічне значення. Якщо resave: true, сеанс зберігається в сховищі щоразу, навіть якщо нічого в req.session не змінилося. Якщо resave: false(рекомендовано), сеанс зберігається лише після його зміни, що зменшує кількість непотрібних записів та покращує продуктивність.
  - saveUninitialized** – логічне значення. Якщо saveUninitialized: true, кожен новий сеанс зберігається в сховищі, навіть якщо до req.session не додано жодних даних. Якщо saveUninitialized: false(рекомендовано), сеанс зберігається лише під час додавання даних, що зменшує непотрібне сховище та покращує конфіденційність.
  - cookie.maxAge, cookie.httpOnly, cookie.secure, cookie.sameSite.**

```

app.listen(3000, () => {
 console.log("Server running on http://localhost:3000");
});

```

# Основи роботи з сеансами. localStorage та sessionStorage

---

```
// Saving data to localStorage
localStorage.setItem('username', 'john_doe');

// Retrieving data from localStorage
let username = localStorage.getItem('username');
console.log(username); // Output: john_doe

// Removing data from localStorage
localStorage.removeItem('username');

// Clearing all data
localStorage.clear();
```

```
// Saving data to sessionStorage
sessionStorage.setItem('userToken', '123456789');

// Retrieving data from sessionStorage
let token = sessionStorage.getItem('userToken');
console.log(token); // Output: 123456789

// Removing data from sessionStorage
sessionStorage.removeItem('userToken');

// Clearing all data
sessionStorage.clear();
```

- Ви можете зберігати налаштування користувача, такі як вибір теми або мовні налаштування, у localStorage, щоб вони зберігалися протягом усіх сеансів:

```
// Saving user preference
localStorage.setItem('theme', 'dark');

// Retrieving user preference
let theme = localStorage.getItem('theme');
if (theme) {
 document.body.classList.add(theme); // Apply the saved theme
}
```

# Основи роботи з сеансами. localStorage та sessionStorage

```
// Saving token during login
sessionStorage.setItem('authToken', 'abcdef12345');

// Using token during the session
let token = sessionStorage.getItem('authToken');
```

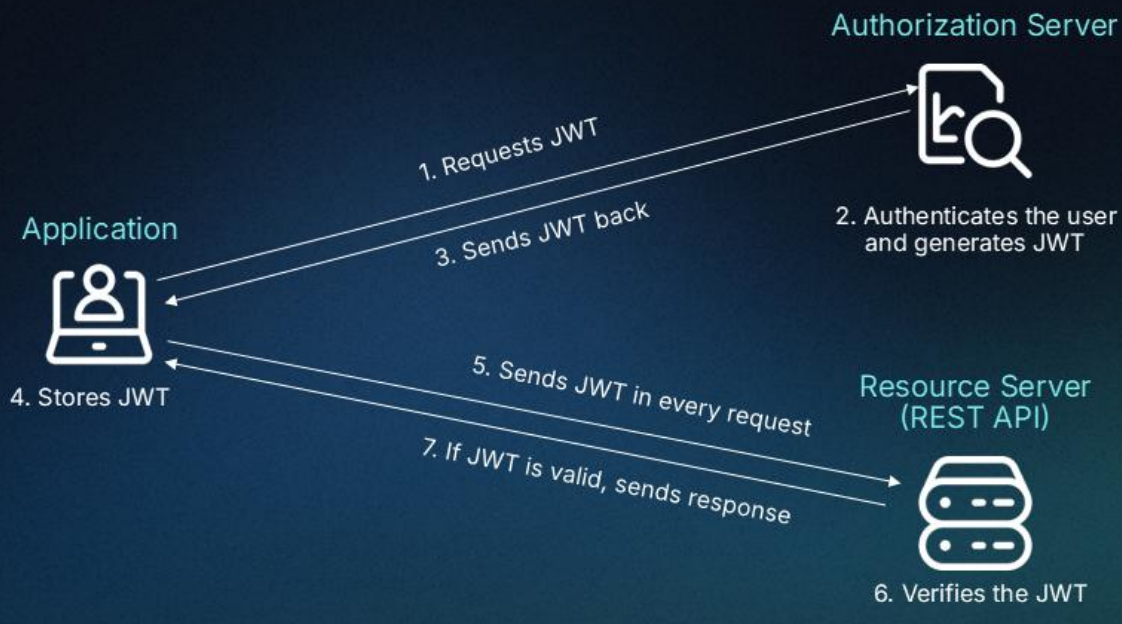
```
// Storing form data
document.querySelector('form').addEventListener('input', function() {
 sessionStorage.setItem('formData', JSON.stringify(getFormData()));
});

// Retrieving form data on page load
window.addEventListener('load', function() {
 let formData = sessionStorage.getItem('formData');
 if (formData) {
 fillForm(JSON.parse(formData));
 }
});
```

- Ви можете використовувати sessionStorage для зберігання тимчасового токена автентифікації, який має бути доступний лише під час сеансу користувача.
- Іноді користувачі можуть випадково оновити сторінку та втратити дані форми.
  - Ви можете використовувати sessionStorage для зберігання даних форми та їх відновлення після оновлення.
- Рекомендовані практики роботи з вебсховищем:
  - Уникайте конфіденційних даних: не зберігайте конфіденційні дані, такі як паролі, номери кредитних карток чи іншу особисту інформацію, у localStorage або sessionStorage.
  - Обов'язково очистіть непотрібні дані за допомогою clear() або removeItem(), щоб уникнути перевантаження сховища.
  - Оскільки вебсховище зберігає дані у вигляді рядків, під час зберігання складних структур даних слід кодувати та декодувати об'єкти за допомогою JSON.stringify() та JSON.parse().

# Вебтокени JSON (JWT)

## JWT Lifecycle



- Вебтокени JSON (JWT) – це компактні, автономні токени, що використовуються для безпечної передачі інформації між сторонами, зберігаючи при цьому цілісність даних.

- Багато постачальників авторизації використовують JWT, щоб забезпечити захищений від несанкціонованого доступу спосіб ідентифікації автентифікованого користувача в додатку та перевірки його авторизованих дій.
- Враховуючи безстанову природу HTTP, після отримання токена застосунком його необхідно зберегти та використовувати для наступних запитів API до сервера.
- Безпечне зберігання JWT часто ігнорується, що може призвести до деяких серйозних вразливостей.
- Наприклад, атаки міжсайтового скриптингу (XSS) можуть красти JWT, дозволяючи зломисникам видавати себе за користувачів та отримувати доступ до конфіденційних даних.

Життєвий цикл JWT складається з трьох етапів:

- 1) сервер авторизації генерує JWT після успішної автентифікації користувача.
- 2) програма зберігає токен, зазвичай у браузері.
- 3) програма включає токен у заголовок авторизації під час надсилання запитів до сервера ресурсів, який перевіряє токен перед обробкою запиту.

# Вебтокени JSON (JWT)

---

- Неправильне зберігання може призвести до крадіжки токенів, що дозволить зломисникам видавати себе за інших користувачів або розширювати їхні привілеї.
- Поширені вектори атак:
  - **Атаки з використанням довільних підписів** трапляються, коли сервери не перевіряють підписи токенів, що дозволяє зломисникам змінювати вимоги та підвищувати привілеї або видавати себе за інших користувачів.
    - Хоча це переважно проблема на стороні сервера, безпечне сховище JWT ускладнює для зломисників отримання дійсного токена для вивчення та маніпулювання.
  - **Алгоритмічна none-атака** відбувається, коли сервери помилково приймають непідписані JWT через те, що параметр alg має значення none.
    - Сховище JWT не повністю пов'язане з цією атакою.
    - Однак зломисникам важче створити шкідливий токен, якщо вони не можуть отримати доступ до дійсного.
  - **Маніпуляція з ідентифікатором ключа (kid)** використовує вразливості в параметрі kid шляхом впровадження шкідливих команд у процес перевірки ключа токена.
    - Як і з іншими векторами атаки, маніпуляція з Kid є більш простою, якщо зломисники отримують дійсний JWT.
    - Однак, вразливість необхідно виправити на стороні сервера.
  - **Атаки методом перебору** спрямовані на слабкі або прості симетричні секрети шифрування, що дозволяє зломисникам генерувати шкідливі токени шляхом вгадування секрету.
    - Якщо токени зберігаються небезпечно, зломисники можуть легко отримати дійсні токени для використання як орієнтир під час спроби вгадати секрет.

# Вебтокени JSON (JWT)

---

- Щоб зменшити ці ризики, вкрай важливо впровадити належну перевірку токенів на стороні сервера, вимкнути незахищені алгоритми, використовувати надійне шифрування та перевірити всі параметри в токені.
  - Зберігаючи токени в клієнтській програмі, ви повинні робити це безпечно, щоб запобігти несанкціонованому доступу до дійсних токенів, які злоумисники можуть вивчити та маніпулювати ними.
- Існує кілька варіантів зберігання токенів на стороні клієнта в браузері.
  - Сеансове або вбудоване сховище може бути ідеальним, якщо сеанси користувачів зазвичай короткі та після їх завершення відбувається вихід із системи.
  - Однак, ви можете розглянути файли cookie або локальне сховище для триваліших сеансів користувачів.

# Вебтокени JSON (JWT). Зберігання в локальному сховищі

```
// Store JWT
localStorage.setItem('jwtToken', response.data.token);

// Use JWT in request
const token = localStorage.getItem('jwtToken');
const response = await fetch('https://example.com/api/data', {
 headers: {
 'Authorization': `Bearer ${token}`
 }
});

// Clear JWT (log out)
localStorage.removeItem('jwtToken');
```

- Простота та надійність в різних сеансах роблять локальне сховище популярним вибором для зберігання JWT у браузері.
  - Однак важливо зазначити, що локальне сховище вразливе до XSS-атак, коли зломисники можуть впроваджувати шкідливі скрипти для доступу до збережених даних, включаючи JWT.
  - Обов'язково впровадьте відповідні заходи безпеки.
- Локальне сховище є популярним вибором для зберігання JWT, оскільки воно дозволяє зберігати токени на різних сторінках і легко доступне з JavaScript.
  - Ліміт пам'яті 5Мб достатній для зберігання токенів.
  - Проте потрібно виправляти XSS-вразливості, а також відсутній механізм автоматичного закінчення терміну дії токенів.

# Вебтокени JSON (JWT). Зберігання в cookies

```
app.post('/login', (req, res) => {
 const token = generateJWT(user);

 // Insert the token into the `auth_jwt` cookie in the response
 res.cookie('auth_jwt', token, {
 httpOnly: true,
 secure: true,
 maxAge: 60 * 60 * 1000 // 1 hour
 });
});
```

Запит із файлом cookie в клієнтському JavaScript-кодi

```
const response = await fetch('/protected', {
 method: 'GET',
 // Set credentials to same-origin so cookies are included
 credentials: 'same-origin'
});
```

- Файли cookie пропонують додаткову безпеку порівняно з іншими варіантами зберігання на стороні клієнта.
  - Вони вимагають модифікацій на стороні сервера для створення безпечних файлів cookie, що містять токен.
  - За правильного налаштування файли cookie менш вразливі до XSS-атак, але вони все ще можуть бути схильні до атак міжсайтової підробки запитів (CSRF).
- Використовуйте прапорець HttpOnly, щоб гарантувати, що клієнтський JavaScript не матиме доступу до файлів cookie.
  - Прапорець Secure гарантує, що файл cookie надсилатиметься лише через безпечний канал, який не вразливий до атак типу «посередник».
  - Атрибут SameSite можна використовувати з іншими стратегіями захисту від CSRF, щоб запобігти атакам CSRF.

# Вебтокени JSON (JWT). Зберігання в cookies

Перевірка токена на сервері за допомогою Express:

```
app.get('/protected', (req, res) => {
 const jwt = req.cookies.auth_jwt;
 if (!verifyToken(jwt)) {
 return res.status(401).json({ message: 'Token is invalid' })
 }

 // Continue processing the request here since the JWT is valid
});
```

Вихід користувача передбачає просте очищення файлу cookie:

```
app.get('/logout', (req, res) => {
 res.clearCookie('auth_jwt');
 return res.status(200).json({ message: 'User logged out.' });
});
```

- Зберігання файлів cookie забезпечує хороший баланс безпеки та функціональності.
  - Однак пам'ятайте про необхідність впроваджувати належні заходи безпеки, такі як використання прапорців HttpOnly, Secure та SameSite, а також механізми захисту від CSRF.
  - JWT автоматично включається до запитів, доступність на вкладках браузера.
- Керування сховищем токенів на стороні сервера дозволяє серверу відкликати токени, якщо це необхідно, перед обробкою запитів.
  - Це може статися, якщо користувач виходить з іншої системи, а на сервер надсилається запит на вихід через зворотний канал.
- Недоліки:
  - Вразливий до CSRF-атак (пом'якшення за допомогою SameSite-атрибутів та механізмів захисту від CSRF).
  - Обмежений обсяг пам'яті (близько 4 КБ).
  - Часто вимагає додаткової логіки на стороні сервера.

# Вебтокени JSON (JWT). Зберігання сеансів

---

```
// Store JWT
localStorage.setItem('jwtToken', token);

// Retrieve JWT
const token = localStorage.getItem('jwtToken');

// Use JWT in request
const response = await fetch('https://example.com/api/data', {
 headers: { 'Authorization': `Bearer ${token}` }
});

// Remove JWT (logout)
localStorage.removeItem('jwtToken');
```

- Сховище сеансів трохи безпечніше, ніж локальне сховище, оскільки воно очищується після закриття сторінки.
  - Однак сховище сесій все ще вразливе до XSS-атак.
  - Використовуйте шпаргалку OWASP та Політику безпеки контенту, щоб запобігти цим атакам.
  - Однак шкідливі розширення браузера все ще можуть отримати доступ до сховища сеансів.
- Переваги:
  - Легко впровадити
  - Доступно для клієнтських скриптів
  - Автоматичне очищення токенів після закриття сторінки
  - Великий ліміт пам'яті ( ліміт 5 МБ )
- Недоліки:
  - Вразливий до XSS-атак
  - Токен втрачається між оновленнями сторінки та вкладками

# Вебтокени JSON (JWT). Зберігання в оперативній пам'яті

```
class TokenService {
 #token = null;

 setToken(newToken) {
 this.#token = newToken;
 }

 getToken() {
 return this.#token;
 }

 clearToken() {
 this.#token = null;
 }
}

const tokenService = new TokenService();

// Store JWT
tokenService.setToken(response.data.token);

// Use JWT in request
const response = await fetch('https://example.com/api/data', {
 headers: {
 // token is the global variable
 'Authorization': `Bearer ${tokenService.getToken()}`
 }
})

// Clear JWT (logout)
tokenService.clearToken();
```

Сховище в оперативній пам'яті (in-memory) зберігає JWT у пам'яті веб-застосунку за допомогою змінної JavaScript або вебворкера, а не зберігає його в сховищі браузера.

- Такий підхід корисний в односторінкових застосунках (SPA), де сторінка оновлюється нечасто.

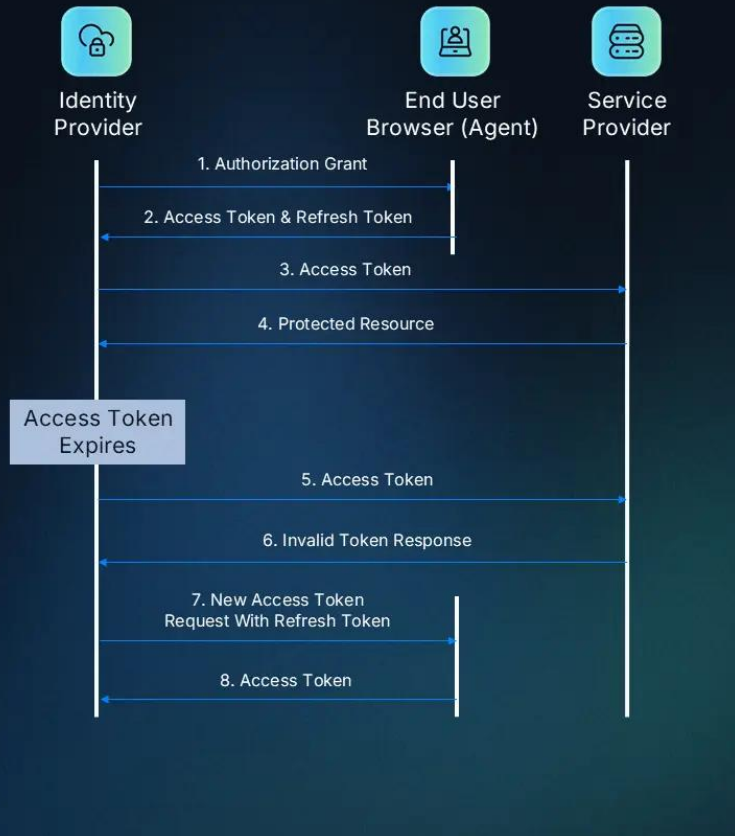
Зберігання в оперативній пам'яті можна вважати дещо безпечнішим, оскільки токен прихований від зловмисника.

- Проявивши достатньо терпіння, зловмисник все ще може з'ясувати, де зберігається токен, і отримати його за допомогою XSS-атаки.
- Зловмисники також можуть спробувати отримати доступ до JWT в оперативній пам'яті за допомогою налагоджувача.
- Подібно до інших методів зберігання, наявність чітко визначеної політики безпеки контенту та впровадження превентивних заходів XSS є життєво важливими.
- Вбудована пам'ять пропонує баланс між безпекою та зручністю. Трохи складніше отримати дані за допомогою XSS-атак, але все одно слід вживати превентивних заходів, щоб запобігти цим атакам.

# Вебтокени JSON (JWT). Рекомендовані практики для зберігання

## Refresh Token Rotation

descope



**Шифрування збережених токенів.** JSON Web Encryption (JWE) – це стандартизований метод шифрування JWT.

- Шифрування відбувається на сервері до того, як токен надсилається клієнту, а ключі шифрування зберігаються на сервері або в спеціальному сервісі керування ключами.
- Оскільки клієнт не має доступу до ключа шифрування, він не може переглянути жодного claim-а в токені.

**Термін дії та ротація токенів.** Термін дії JWT визначається терміном дії токена.

- Після закінчення терміну дії токена необхідно отримати новий за допомогою токена оновлення.
- Токен оновлення – це токен, що видається разом з оригінальним JWT та дозволяє запитувати новий JWT після закінчення терміну дії поточного.
- Під час видачі нового JWT за допомогою токена оновлення сервер авторизації спочатку перевіряє, чи сеанс користувача все ще активний, а потім надсилає новий токен, якщо він активний.
- Ви можете ще більше підвищити безпеку вашого JWT та токена оновлення, ротуючи токени оновлення.
- Щоразу, коли ви запитуєте новий JWT за допомогою токена оновлення, разом із ним генерується та повертається новий токен оновлення.
- Повернений токен оновлення – єдиний дійсний токен, який можна використовувати для отримання наступного JWT.
- Також слід встановити термін дії для ваших токенів оновлення для безпеки. Навіть якщо зломисники зможуть отримати доступ до токена оновлення, вони можуть не використати його вчасно.

# Вебтокени JSON (JWT). Рекомендовані практики для зберігання

---

- **Обробка відкликання токенів.** Якщо сервер використовує лише заявку на термін дії JWT, щоб визначити, чи минув термін дії токена, то токен все ще може бути дійсним після виходу користувача з системи або блокування його в програмі.
  - Хоча скорочення часу дії JWT може допомогти мінімізувати ризик, все ще існує певний проміжок часу, протягом якого токен дійсний після завершення сеансу користувача.
  - Реалізувавши blocklist, сервер може використовувати час закінчення терміну дії токена та сам blocklist, щоб визначити, чи токен дійсний.
  - Коли користувач виходить із системи або його обліковий запис заблоковано, сервер авторизації надсилає запит на вихід через зворотний канал іншим серверам, що вказує на завершення сеансу цього користувача.
  - Кожен сервер може використовувати цю подію для оновлення свого blocklist даними користувача.
  - Потім, коли інший запит надсилається з використанням JWT цього користувача, він може бути визнаний недійсним, навіть якщо термін його дії ще не минув.
  - Хоча більшість програм нормально працюють з JWT, що триває трохи довше, ніж сеанс користувача, деякі можуть вимагати суворішого захисту.
  - Реалізація відкликання токенів дозволяє отримати переваги JWT, пропонуючи ефективний спосіб негайного блокування токенів.

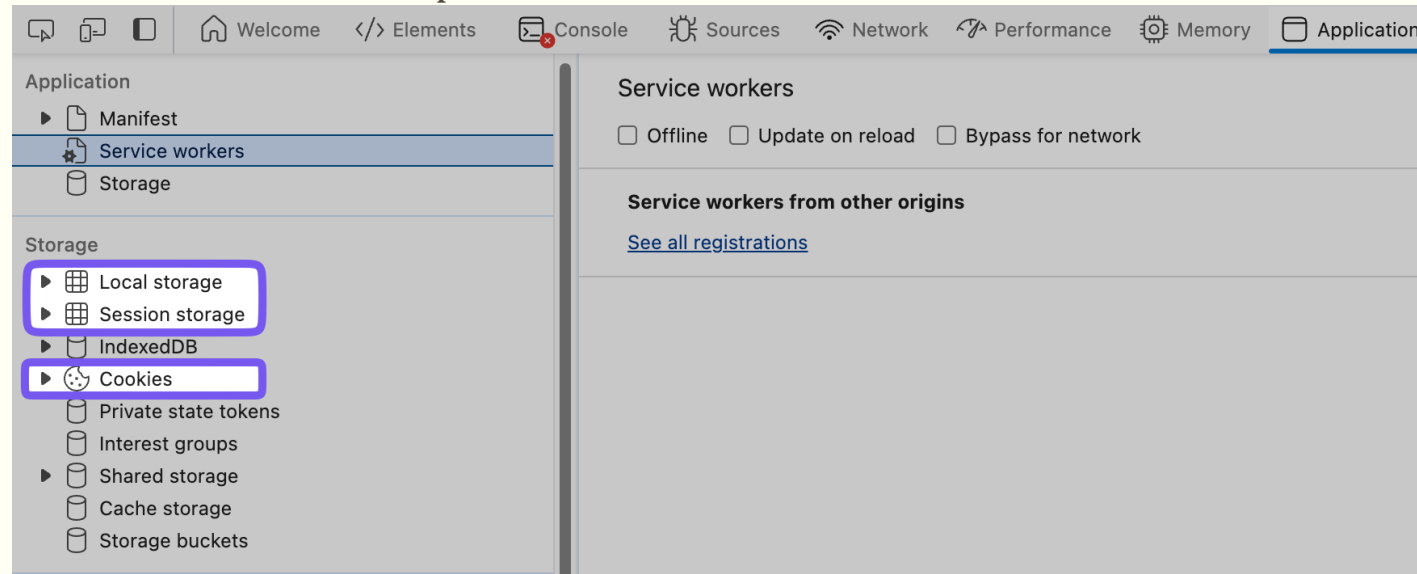
# Вебтокени JSON (JWT). Рекомендовані практики для зберігання

---

- Захист від XSS та CSRF атак.
  - Кожен з розглянутих методів зберігання може бути схильним до XSS- та CSRF-атак.
  - Щоб запобігти XSS-атакам, використовуйте автентифікацію за допомогою cookie, щоб JavaScript не мав доступу до JWT.
  - Однак, якщо вашому фронтендному JavaScript потрібен доступ до JWT, ви можете впровадити політику безпеки контенту, щоб обмежити, які скрипти можуть виконуватися на сторінці.
  - Якщо ви використовуєте автентифікацію за допомогою файлів cookie, переконайтеся, що ваші файли cookie налаштовано правильно.
  - Вони повинні бути позначені HttpOnly та Secure. Крім того, налаштуйте SameSite-обмеження, коли файли cookie надсилаються в запитах від сторонніх URL-адрес.
  - Інші механізми захисту від CSRF, такі як токени захисту від підробки, також можуть допомогти захиститися від CSRF-атак.
  - Не існує ідеального рішення, коли йдеться про атаки XSS та CSRF.
  - Оцініть свої вимоги, вирішіть, який механізм зберігання даних ви використовуватимете, а потім врахуйте вразливості, пов'язані з цим методом.

# Вебтокени JSON (JWT). Виправлення поширених проблем

- Усунення несправностей JWT може бути складним завданням.
  - Якщо вам важко зрозуміти, чи успішно зберігається JWT, скористайтесь вкладкою «Application» в інструментах розробника вашого браузера, щоб проаналізувати локальне сховище, сховище сеансу та файли cookie, пов'язані з поточною сторінкою.



- Якщо токен відображається у вибраному вами механізмі зберігання, перевірте, чи він має правильний формат. Він має складатися з трьох частин, розділених крапками.
- Ви також можете відстежувати життєвий цикл повідомлень журналу токенів. Це може допомогти визначити, де у вашому коді токен обробляється або зберігається неправильно.

# Вебтокени JSON (JWT). Виправлення поширених проблем

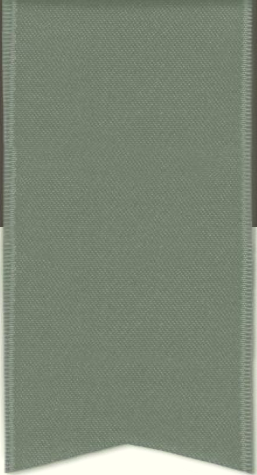
---

- Термін дії токенів закінчується, тому переконайтеся, що ви реалізували токени оновлення та використовуєте їх для отримання нових токенів, коли термін дії старих закінчується.
  - Під час використання токенів оновлення краще проактивно оновлювати токен до закінчення терміну його дії, відстежуючи час його закінчення та запускаючи оновлення за кілька секунд до закінчення терміну дії.
  - В іншому випадку ви можете надіслати токен, який все ще дійсний протягом частки секунди на момент надсилання запиту, але його термін дії закінчується, коли сервер його отримує.
  - Якщо запит на оновлення не вдається, спробуйте ще раз, оскільки причиною може бути проблема з мережею.
  - Якщо знову не вдається, перенаправте користувача на екран автентифікації з повідомленням про те, що для продовження використання програми необхідно увійти в систему.
  - Це підвищить стійкість вашої програми до помилок автентифікації.
- Що робити, якщо вашій програмі потрібно надсилати запити до кількох доменів, використовуючи один і той самий токен?
  - Наприклад, фронтенд програми може бути на іншому домені, ніж API.
  - Слід розглянути можливість використання бекенд-для-фронтенду (BFF) для вашої програми.
  - BFF об'єднує всі ці виклики API до різних сервісів в один сервіс. Ваш cookie потрібно налаштувати лише для домену вашого BFF. BFF потім може проксити запит до відповідного сервісу за допомогою JWT.

# Вебтокени JSON (JWT). JWT-сховище на мобільному пристрої

---

- Багатьом додаткам потрібен доступ до обмежених API, які потребують автентифікації.
  - Рішення для зберігання JWT, такі як файли cookie з HttpOnly, добре працюють у вебзастосунку, але є складнішими в мобільних застосунках, враховуючи відсутність специфічних для браузера механізмів, таких як керування файлами cookie.
  - Вашому застосунку потрібно буде вручну зберігати файл cookie та додавати його до кожного запиту до API.
  - Під час зберігання файлу cookie також потрібно переконатися, що він безпечний та недоступний для інших застосунків або кінцевого користувача.
- Мобільні платформи пропонують безпечне сховище для зберігання конфіденційних секретів, таких як JWT.
  - На Android ви можете використовувати Keystore для шифрування JWT, які потім можна зберігати за допомогою SharedPreferences.
  - Аналогічно, iOS дозволяє використовувати Keychain для шифрування та зберігання секретів на пристрої користувача. Використання цих сервісів схоже на те, як ви зберігаєте JWT у локальному сховищі в Інтернеті: після входу користувача в систему, отримайте JWT та збережіть його.
  - Пам'ятайте, що ці пристрої більш схильні до крадіжки, тому створюйте короточасні JWT, які часто ротуються за допомогою токенів оновлення.
  - Ви навіть можете зберігати ідентифікатор сеансу замість JWT у мобільному додатку.
  - За допомогою ідентифікатора сеансу додаток надсилає запит до сервера, який виконує пошук у базі даних, щоб отримати пов'язаний JWT та перевірити його.



# ДЯКУЮ ЗА УВАГУ!

Наступне питання: Інструменти React для керування станом